End-to-End Communication System Simulation

Automotive Key-Fob (Remote Keyless Entry) Link with a PCM Payload

Practical 8 – Mini-Project Report

Group members: *[add names and student numbers here]*

October 16, 2026

Abstract

We simulate a complete digital communication chain — source → sampling → 8-bit PCM → packet framing (device ID, KeeLoq-style rolling code, CRC-16) → Manchester line coding → OOK modulation → path-loss/fading/AWGN channel → synchronisation and non-coherent demodulation → decoding → reconstructed signal — in the setting of an automotive key fob. The simulated bit error rate agrees with theory to within Monte-Carlo accuracy. The reconstruction MSE of the 300 Hz test message equals the quantization floor of 6.31×10^{-6} (SQNR = 48.99 dB) for a channel SNR above about 7.6 dB (in-band) and degrades within a few dB below it. A 90 % packet success rate needs an in-band SNR of 6.9 dB, which is 14.2 dB above the Shannon limit for the same rate and bandwidth. The rolling-code layer rejects replayed, forged and unpaired packets in an end-to-end waveform simulation, and a link budget predicts a 90 % range of 40 m (AWGN), 30 m (Rician) and 22 m (Rayleigh fading).

Contents

1 Introduction and project idea **3**

2 System overview **3**

3 The chain stage by stage **4**

3.1 Source and sampling 4

3.2 Quantization and PCM 5

3.3 Packet framing, CRC and rolling code 6

3.4 Line coding 6

3.5 Digital modulation: OOK 7

3.6 Channel 8

3.7 Demodulation, synchronisation and decoding 8

3.8 PCM decoding and reconstruction 10

4 Performance analysis: channel SNR versus reconstruction quality **11**

4.1 Bit error rate 11

4.2 MSE of the reconstructed signal 11

5 Link to Shannon's channel-capacity theorem **13**

6 Security layer: rolling code **14**

7 Range: from SNR to distance **15**

8 Verification, assumptions and limitations **16**

9 Conclusion **17**

1 Introduction and project idea

A remote keyless entry (RKE) fob is a battery-powered transmitter that sends a short radio packet to the car. The packet contains a device identifier, a command (lock, unlock, trunk, panic) and a *rolling code*, a counter that is encrypted so that every transmission is different and a recorded packet cannot be replayed. Because the battery must last for years, fobs use the simplest modulation that works, *on-off keying* (OOK, a form of ASK), at 315 or 433.92 MHz, together with Manchester line coding.

The lecturer's guideline asks for the chain *Source* → *Sampling* → *Quantization/PCM* → *Line coding* → *Digital modulation* → *AWGN channel* → *Demodulation* → *Decoding* → *Reconstructed signal*, a performance analysis of channel SNR against reconstruction MSE, and a link to Shannon's theorem. A key fob normally sends only a few control bytes, so to exercise *every* block of the guideline we let the fob packet carry an optional *telemetry / voice-snippet field*: the 50 ms, 300 Hz test message of the guideline (8 kHz sampling, 8-bit PCM). The PCM bits are the payload of the fob packet and everything else (framing, rolling code, OOK, realistic channel) is what makes the system a key fob rather than a textbook BPSK link.

What we added to the textbook chain. (i) a real packet format with preamble, sync word, device ID, command, rolling code and CRC-16; (ii) OOK with non-coherent detection instead of BPSK (a BPSK/NRZ curve is still simulated as a reference); (iii) blind timing recovery by correlating with the known preamble and sync word; (iv) a KeeLoq-style rolling-code state machine with replay, forgery and re-synchronisation tests; (v) a link budget with path loss and Rayleigh/Rician fading that turns SNR into distance.

Table 1: Where each step of the guideline is implemented (Python, folder `functions/`).

Guideline step	What we do	Code
Message signal	$\sin(2\pi \cdot 300\,t)$, 50 ms, "analog" grid of 64 kHz	`source.py`
Sampling	$F_s = 8$ kHz, 400 samples	`source.py`
Quantization	uniform mid-rise, $n = 8$ bit, 256 levels	`quantizer.py`
PCM	natural binary, MSB first, 3200 bits	`pcm.py`
Line coding	Manchester (NRZ-L shown for comparison)	`line_coding.py`
Digital modulation	OOK on a scaled carrier (BPSK as reference)	`modulation.py`
AWGN channel	`awgn(x,SNR,'measured')` equivalent, plus path loss and fading	`channel.py`
Demodulation	mixing, integrate-and-dump, timing sync, energy detector	`demodulation.py`, `receiver.py`
Decoding	Manchester, frame parser, CRC, rolling code, PCM decoder	`framing.py`, `rolling_code.py`
Reconstruction	D/A (hold) + 6th-order low-pass filter	`reconstruction.py`
MSE/SNR analysis	Monte-Carlo sweeps, theory	`analysis.py, metrics.py`
Entry point	runs everything, saves all figures	`main.py`

2 System overview

Figure 1 shows the complete system. The transmitter is the fob, the receiver is the car. Table 2 lists the parameters.

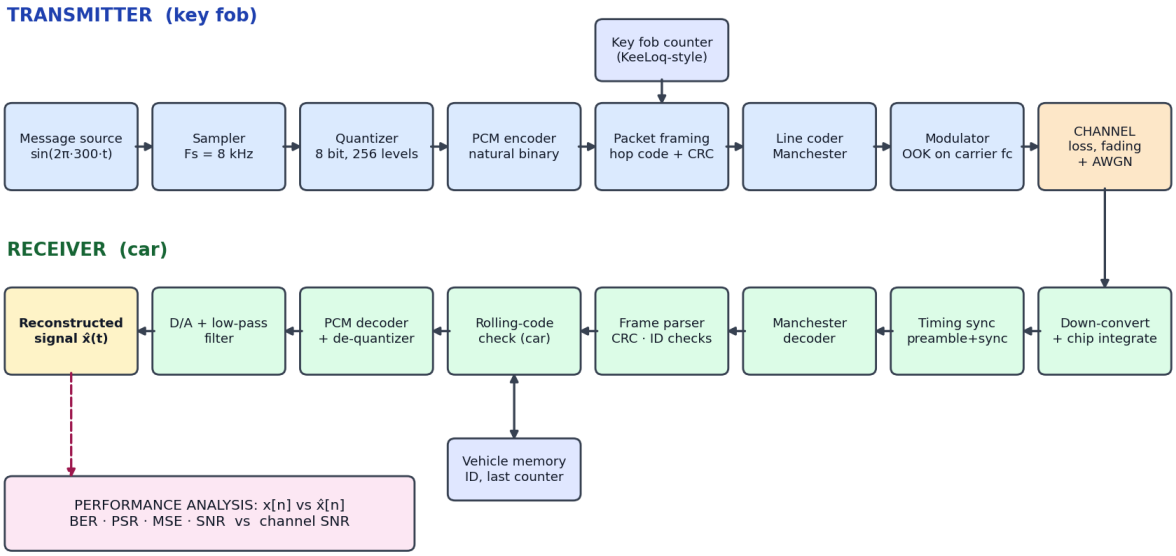


Figure 1: Block diagram of the complete simulated system (top: key fob, bottom: car).

Table 2: System parameters (all in `functions/config.py`).

Parameter	Value	Comment
Message	300 Hz sine, amplitude 1, 50 ms	guideline values
Sampling rate F_s	8 kHz (400 samples)	15 message periods
PCM	8 bit $\Rightarrow$ 3200 payload bits	$\Delta = 0.00781$
Packet	3320 bits (Fig. 5)	346 ms on air
Bit rate R_b	9.6 kbit/s	typical RKE value
Line code	Manchester, 19.2 kchip/s	16 samples per chip
Modulation	OOK, scaled carrier $f_c = 38.4$ kHz	real fobs: 315/433.92 MHz
Simulation rate	$f_{s,rf} = 307.2$ kHz	8 samples per carrier cycle
RF bandwidth B	$4R_b = 38.4$ kHz (null-to-null)	Manchester $\times$ OOK
Channel	path loss, block fading, AWGN	Sec. 7
Monte Carlo	400 packets per SNR point (1.33 Mbit)	seed 2026

Why a scaled carrier. Simulating 433.92 MHz would need a sampling rate of several GHz. The error performance of a pass-band link depends on E_b/N_0 and not on f_c as long as the carrier is much faster than the bit rate, so we use $f_c = 38.4$ kHz, an integer number of carrier cycles per chip. The path-loss model in Sec. 7 still uses the real 433.92 MHz.

3 The chain stage by stage

All figures of this section come from one representative packet sent at a channel SNR of +2 dB (unless stated otherwise).

3.1 Source and sampling

The message is $x(t) = \sin(2\pi f_m t)$ with $f_m = 300$ Hz, drawn on a fine 64 kHz grid to play the role of the analog signal. The ideal sampler keeps every eighth point ($F_s = 8$ kHz, Nyquist frequency 4 kHz $\gg f_m$, so no aliasing and no anti-alias filter is needed).

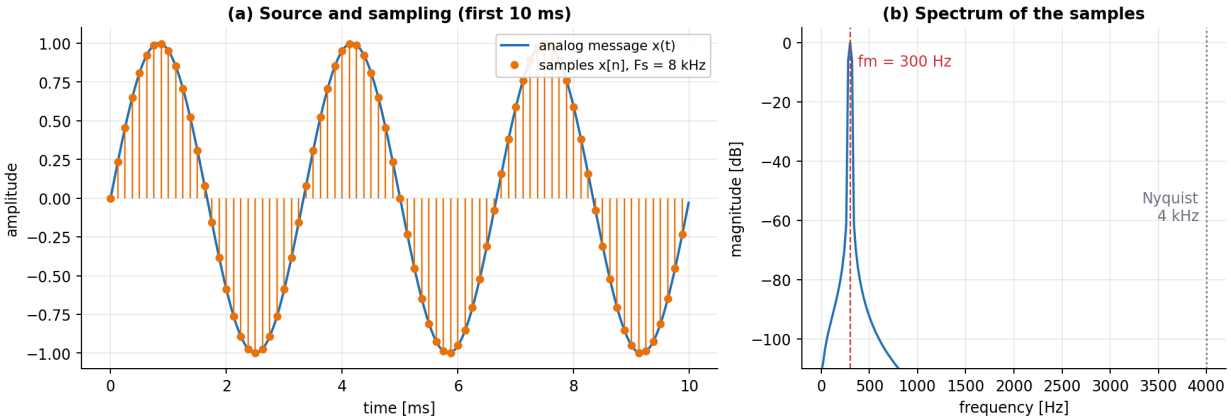


Figure 2: (a) Analog message and its samples. (b) Spectrum of the samples: a single line at 300 Hz.

3.2 Quantization and PCM

The uniform mid-rise quantizer has $L = 2^n = 256$ levels over $[-1, 1]$, step $\Delta = 2/256 = 0.00781$ and error $|e| \leq \Delta/2$. For a full-scale sine the theoretical signal-to-quantization-noise ratio is $\text{SQNR} = 6.02\,n + 1.76 = 49.92$ dB. We *measure* 48.99 dB ($\text{MSE} = 6.31 \times 10^{-6}$): the 400 samples contain only 80 distinct phases (300 Hz at 8 kHz repeats every 3 periods), so the error is not perfectly uniform and lands about 1 dB below the textbook value. This is the **quantization floor** of the whole system: no channel, however good, can push the MSE below it.

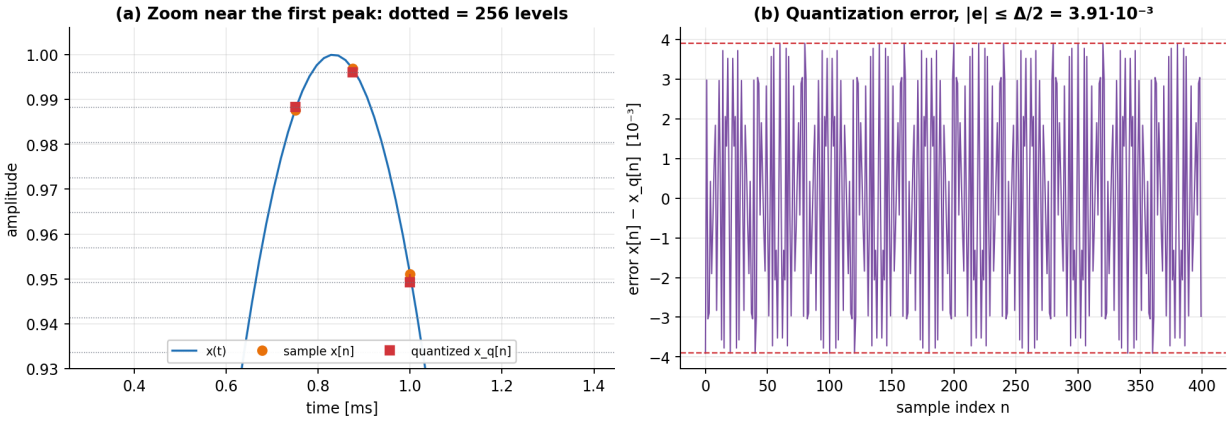
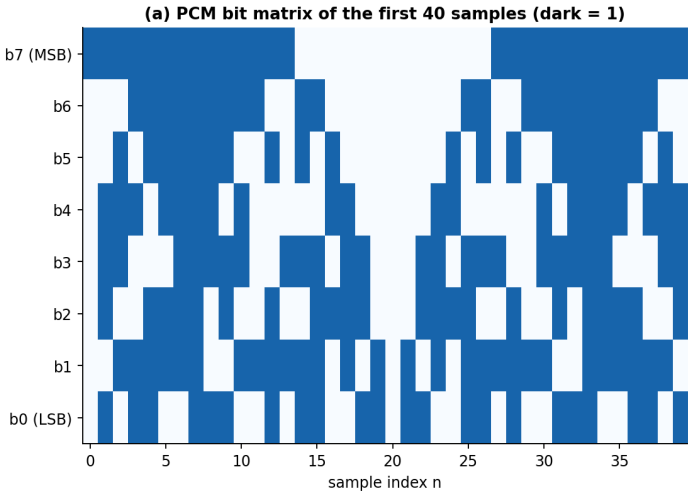


Figure 3: (a) Zoom near the first peak: samples (orange) are moved to the nearest of the 256 levels (red). (b) Quantization error over all samples.

Every level index is written as an 8-bit natural-binary word, MSB first (`pcm.py`); the 400 words give the 3200-bit payload.



(b) Code words (natural binary, MSB first)

n	x[n]	level	PCM word
0	+0.0000	128	10000000
1	+0.2334	157	10011101
2	+0.4540	186	10111010
3	+0.6494	211	11010011
4	+0.8090	231	11100111
5	+0.9239	246	11110110
6	+0.9877	254	11111110
7	+0.9969	255	11111111

Figure 4: PCM bit matrix and the first code words. The MSB (top row) toggles with the sign of the message, the LSB looks random.

3.3 Packet framing, CRC and rolling code

The payload is wrapped in the fob frame of Fig. 5. The 16-bit preamble (`0xAAAA`) is a clock/gain training pattern, the sync word (`0x2DD4`) marks the start of the frame, the 32-bit device ID identifies the fob, the command byte selects lock/unlock/trunk/panic, the 32-bit *hop code* is the encrypted rolling counter (Sec. 6) and the CRC-16/CCITT ($x^{16} + x^{12} + x^5 + 1$, init `0xFFFF`, check value `0x29B1` for "123456789") protects everything from the ID to the end of the payload.

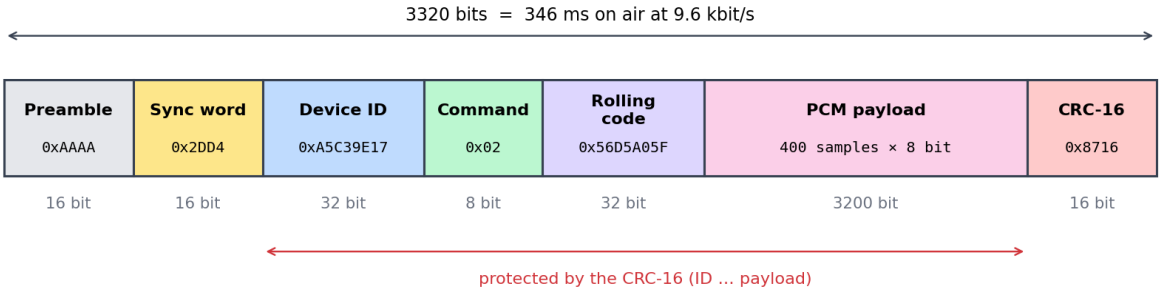


Figure 5: Packet format (field widths not to scale). The example contents are those of the simulated packet.

3.4 Line coding

A *chip* is one line-code symbol. NRZ-L sends one chip per bit; long runs of equal bits give no transitions, a DC component and no clock information. Manchester sends two chips per bit, $1 \to (1, 0)$ and $0 \to (0, 1)$, which guarantees a transition in every bit (self-clocking), a 50 % duty cycle and — crucial for OOK — a decision that compares the two halves of a bit and therefore needs *no threshold*, so it keeps working when path loss changes the received amplitude. The price is twice the bandwidth.

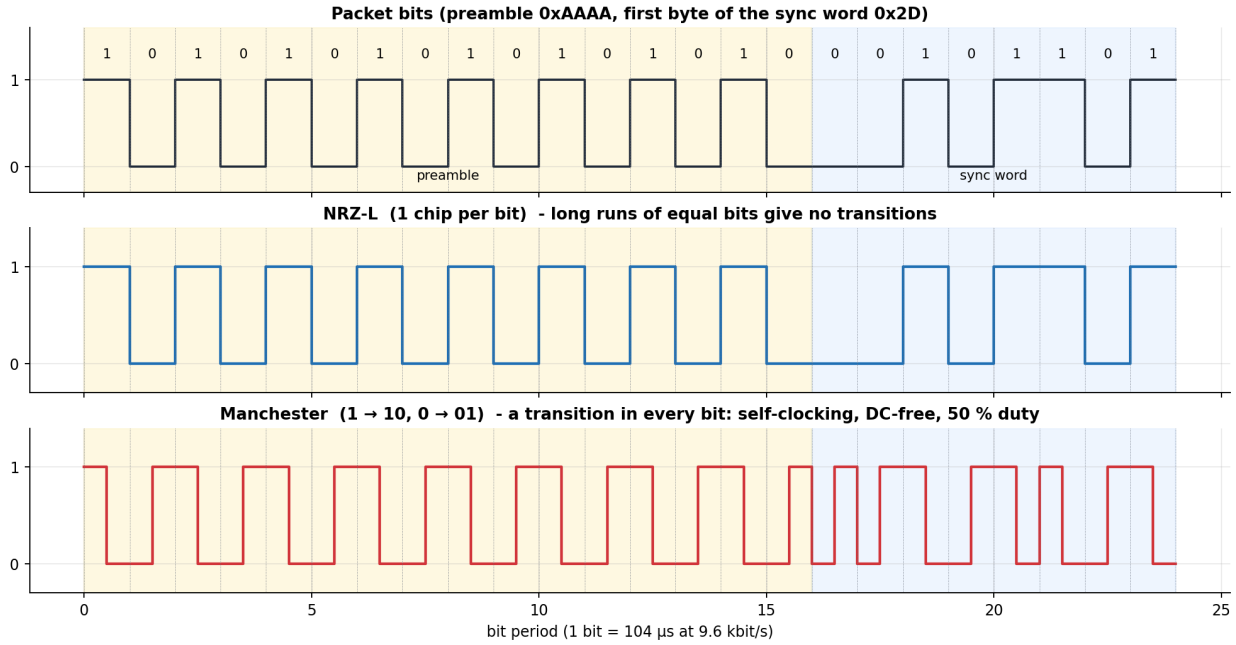


Figure 6: Packet bits, NRZ-L and Manchester waveforms for the preamble and the first byte of the sync word.

3.5 Digital modulation: OOK

With chip $c(t) \in \{0, 1\}$ the transmitted signal is $s(t) = A\,c(t)\cos(2\pi f_c t)$. Because exactly one of the two chips of every Manchester bit is "on", the average power is $P = A^2/4$ and the average energy per bit is $E_b = A^2 T_b/4$. The spectrum (Fig. 7c) shows a carrier line, the Manchester null around it and nulls at $f_c \pm 2R_b$: the null-to-null bandwidth is $B = 4R_b = 38.4$ kHz.

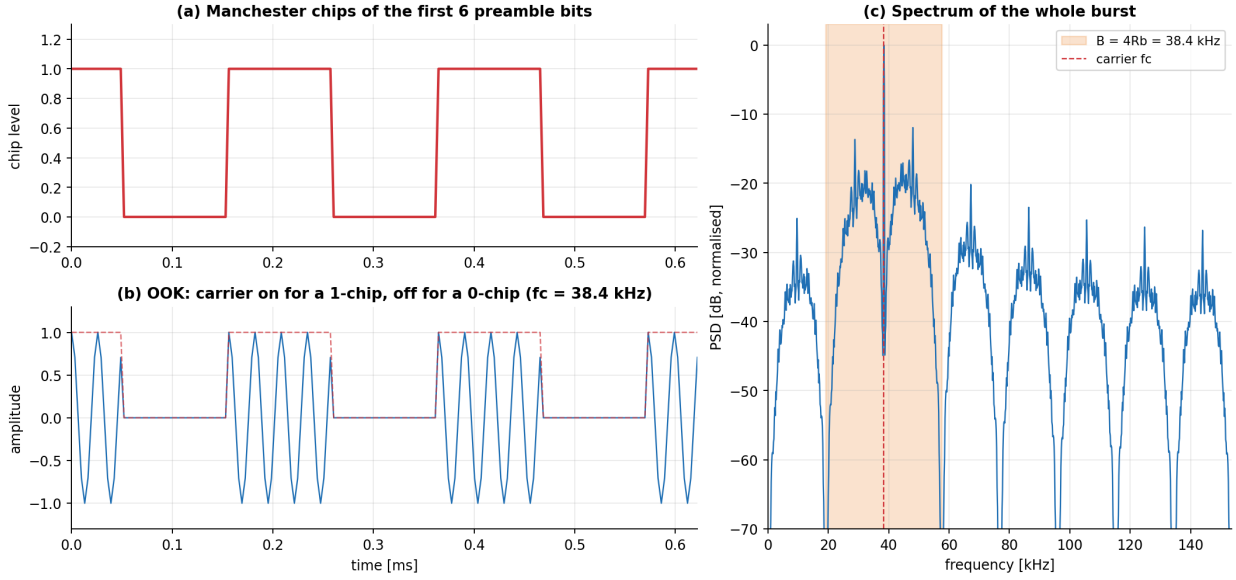


Figure 7: (a) Chips of the first six preamble bits, (b) the OOK burst, (c) power spectral density of the whole burst.

3.6 Channel

The signal is placed at a random start time in an idle record (the receiver does not know when the fob transmits), multiplied by a complex flat-fading gain h (for the AWGN experiments $|h| = 1$ with a random carrier phase, which the receiver does not know) and white Gaussian noise is added. We follow MATLAB's `awgn(tx,SNR,'measured')`:

$$\mathrm{SNR} = \frac{P_{\mathrm{burst}}}{\sigma^2}, \qquad N_0 = \frac{2\sigma^2}{f_{s,rf}}, \qquad \frac{E_b}{N_0} = \mathrm{SNR} \cdot \frac{f_{s,rf}}{2R_b} \quad \Rightarrow \quad \frac{E_b}{N_0}\,[\mathrm{dB}] = \mathrm{SNR}\,[\mathrm{dB}] + 12.04\ \mathrm{dB}. \quad (1)$$

All SNR values in this report are this "measured" SNR unless marked *in-band*. The noise of the simulation occupies $0 \ldots f_{s,rf}/2 = 153.6$ kHz; the same noise power measured only inside the signal bandwidth B is 6.02 dB *higher in SNR* (in-band SNR = SNR +6.02 dB). The in-band SNR is the one that belongs in Shannon's formula.

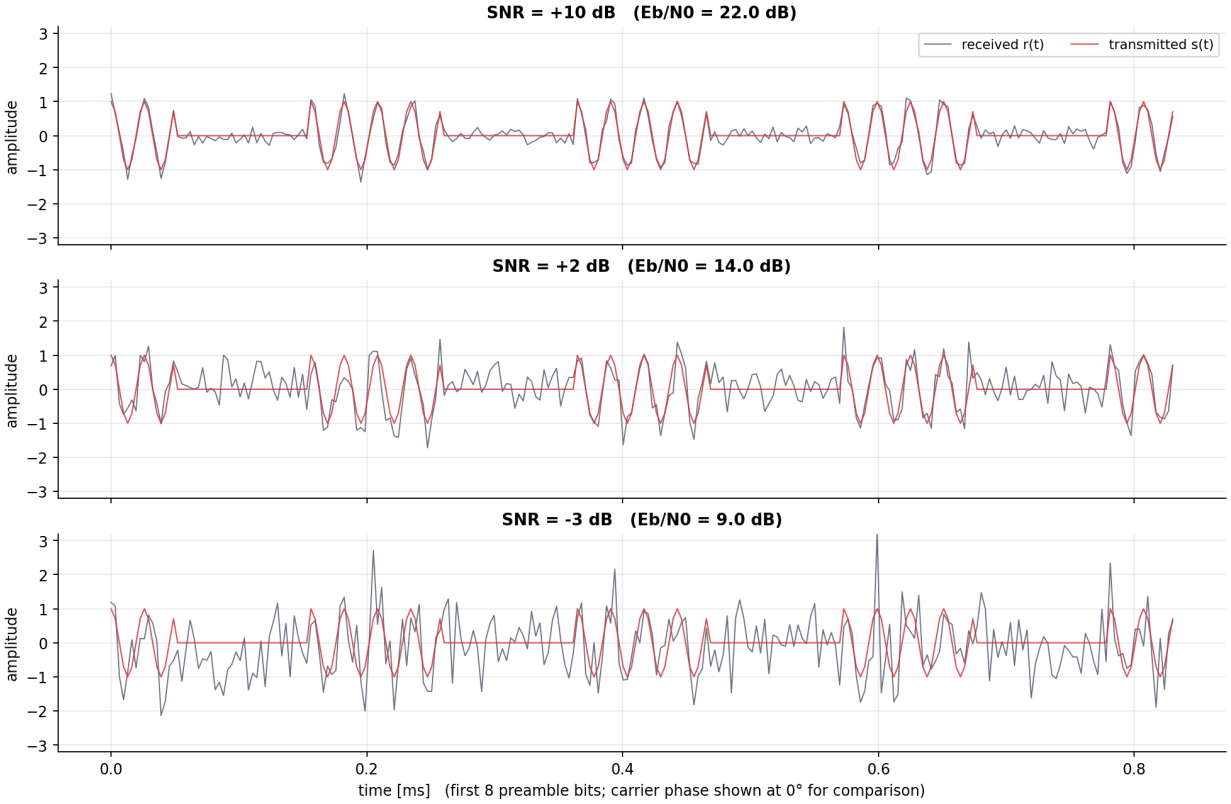


Figure 8: Transmitted (red) and received (gray) signal for three SNRs. For this plot the carrier phase is set to 0° so that the two can be compared.

3.7 Demodulation, synchronisation and decoding

Front end. The record is mixed with $e^{-j2\pi f_c t}$ and passed through a sliding integrate-and-dump over one chip: $y[n] = \sum_{k=0}^{15} r[n+k]e^{-j\omega(n+k)}$. Because 16 samples span an integer number of carrier cycles, the double-frequency term cancels exactly and $E[n] = |y[n]|^2$ is the energy of the chip starting at sample n, *independent of the carrier phase* (non-coherent detection, as in real fobs).

Timing recovery. The receiver does not know where the burst starts. It correlates $E[n]$ with the known ± 1 pattern of preamble+sync (zero mean, so the noise floor cancels) and takes the maximum over all candidate start samples (Fig. 9b).

Decision. For every bit the energies E_1, E_2 of the two chips are compared: bit $= 1$ if $E_1 > E_2$ (Fig. 9c,d). The bits go to the frame parser which checks the sync word (up to 2 errors tolerated), the CRC-16 and the device ID, and then to the rolling-code check.

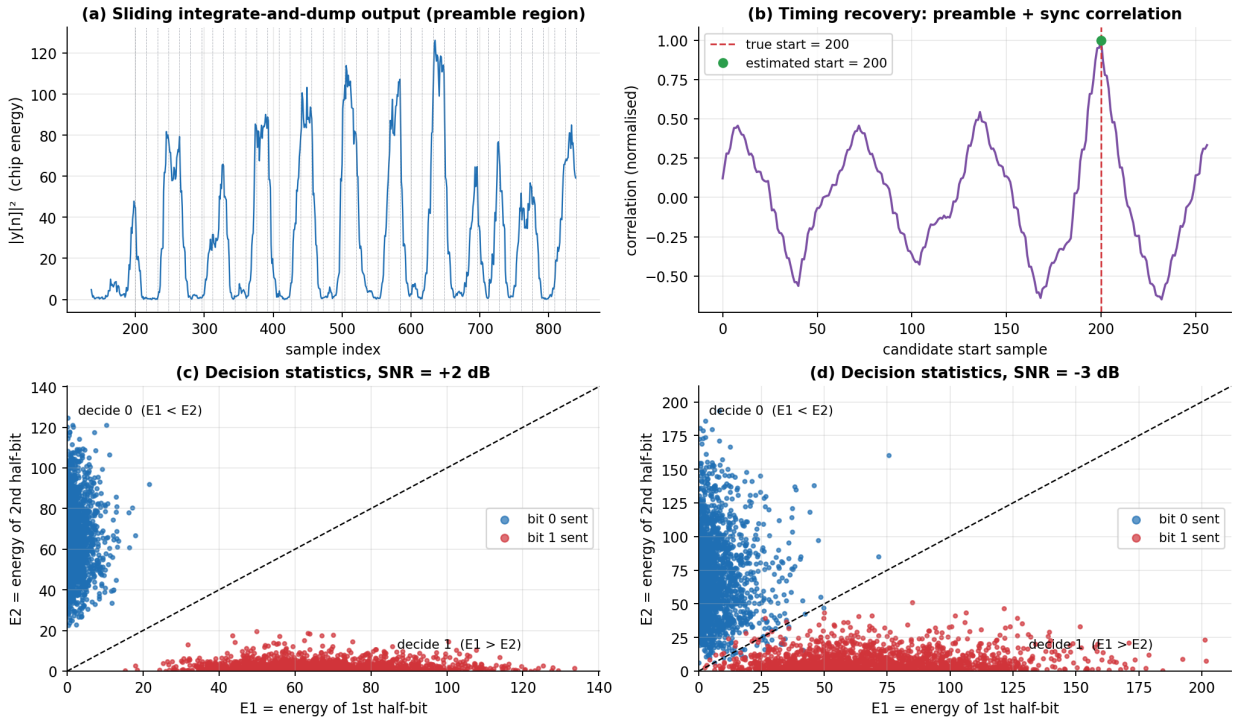


Figure 9: (a) Chip energy of the preamble. (b) Timing recovery. (c,d) Decision plane at +2 dB and −3 dB.

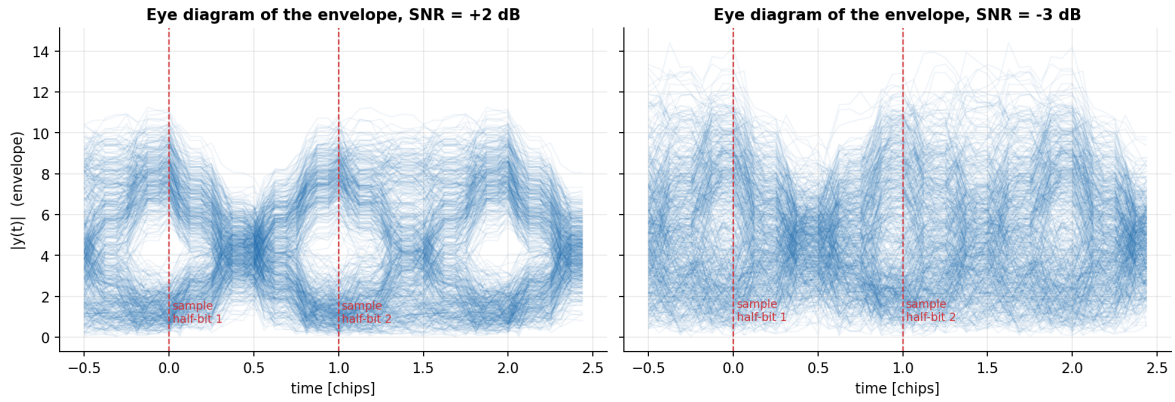
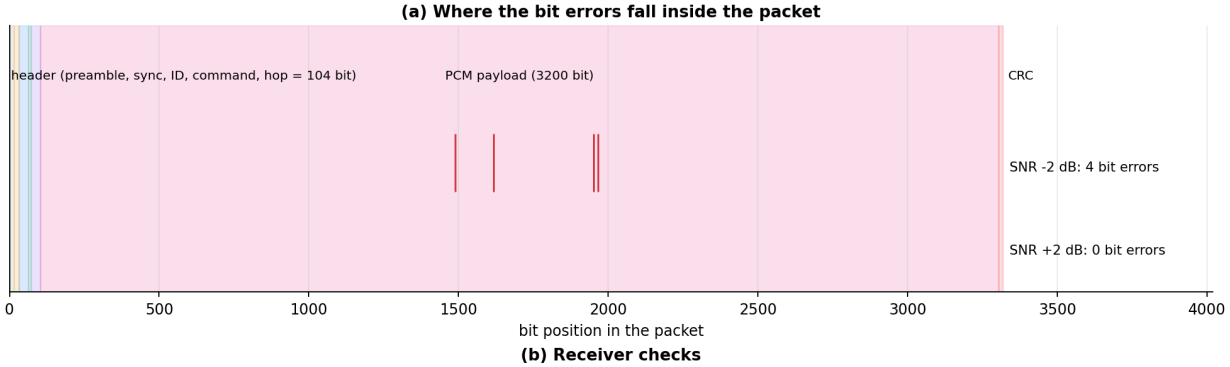


Figure 10: Eye diagram of the received envelope; the sampling instants are the red lines. The eye closes as the SNR drops.



SNR	sync word	CRC-16	device ID	link-layer decision
+2 dB	OK	OK	OK	ACCEPT → rolling-code check
-2 dB	OK	FAIL	OK	DROP packet

Figure 11: Frame parser: error positions and integrity checks for a good and a corrupted packet. A few errors in the payload are enough to fail the CRC and drop the packet.

3.8 PCM decoding and reconstruction

The payload bits are regrouped into 8-bit words, converted to the mid-point level of each quantization interval and sent through a D/A converter modelled as a zero-order hold followed by a 6th-order Butterworth low-pass at 3.4 kHz (zero-phase, half-sample hold delay compensated). The **reconstruction MSE** is measured on the samples, $\text{MSE} = \frac{1}{N}\sum_n (x[n] - \hat{x}[n])^2$, and the reconstruction SNR is $10\log_{10}(P_x/\text{MSE})$ with $P_x = 0.5$.

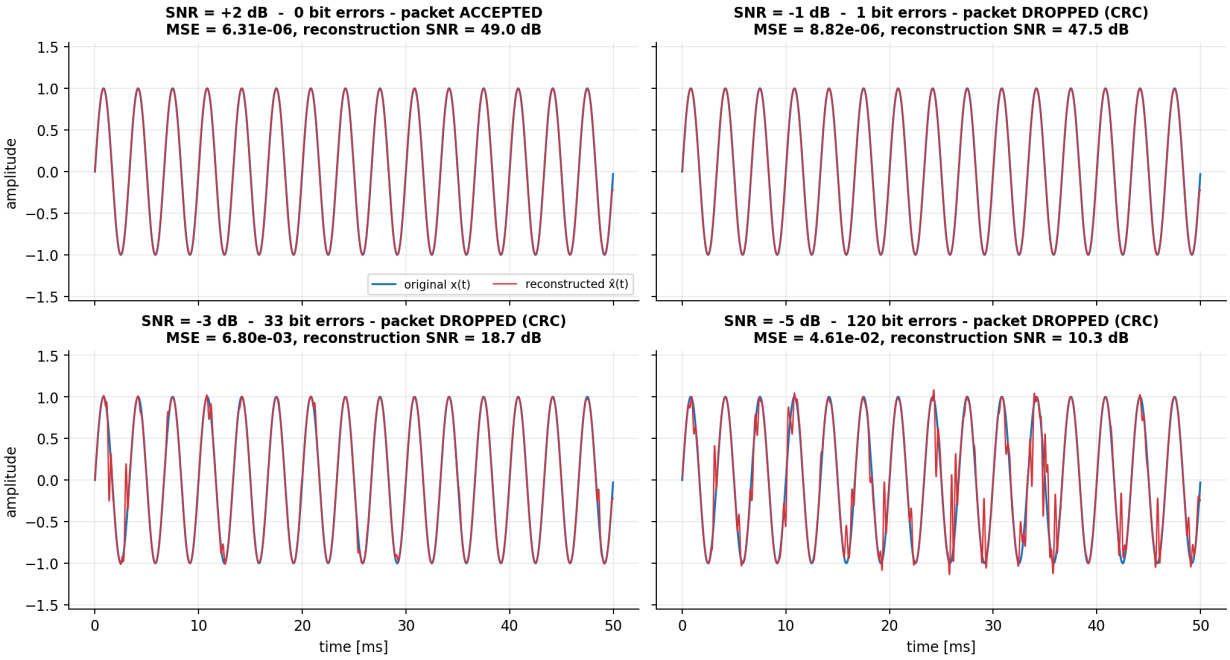


Figure 12: Original and reconstructed message at four channel SNRs (the receiver output *before* the CRC decision, i.e. the raw decoded payload). Single bit errors in the most significant bits appear as large spikes.

4 Performance analysis: channel SNR versus reconstruction quality

For each of 19 SNR values from −8 to +20 dB we sent 400 complete packets (1.33 Mbit per point) through the whole chain with a random start time and random carrier phase, using the real (blind-timing) receiver. Table 3 summarises the results; all curves are in the figures that follow.

4.1 Bit error rate

For non-coherent detection of two orthogonal "on-chip" hypotheses with E_b the average energy per bit, $P_b = \frac{1}{2}\exp(-E_b/2N_0)$; for the textbook coherent BPSK reference $P_b = Q(\sqrt{2E_b/N_0})$. Figure 13 shows that the simulation follows theory over four decades, which validates the modulator, channel, SNR bookkeeping, front end and detector. OOK needs about 3.8 dB more E_b/N_0 than BPSK for the same BER, the price of cheap non-coherent hardware.

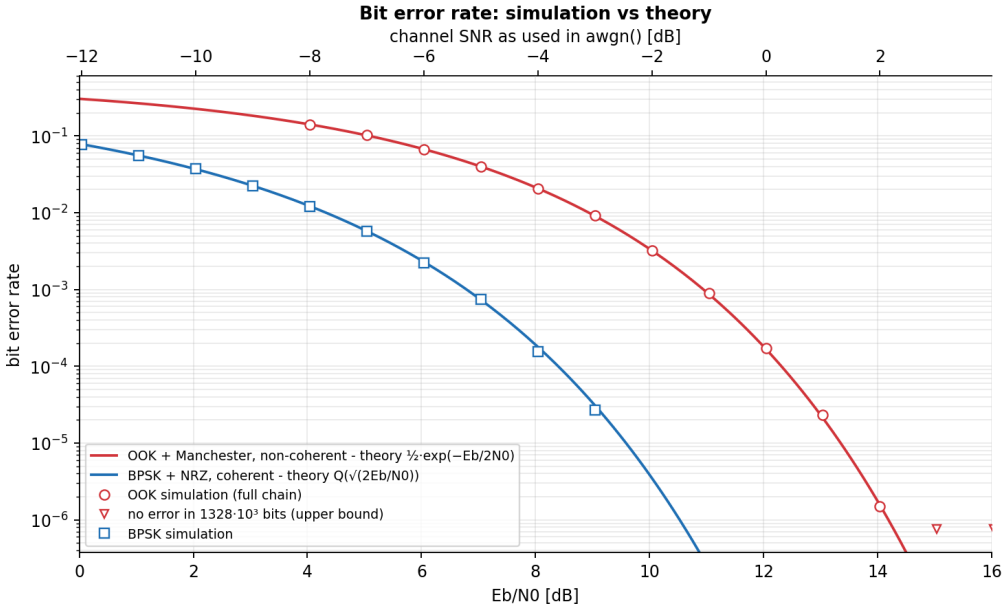


Figure 13: BER versus E_b/N_0 (bottom axis) and the channel SNR used by `awgn` (top axis). Markers: Monte-Carlo, lines: theory. Down-triangles: no error seen (upper bound $1/N_{\text{bits}}$).

4.2 MSE of the reconstructed signal

We report three quantities. *Raw*: the PCM payload is decoded whatever the receiver believes (this is the MSE asked for in the guideline). *Delivered*: the car mutes the output (silence, MSE $= P_x = 0.5$) whenever sync/CRC/ID fail, which is what a real receiver would do. *Theory*: the BER from theory is converted to an MSE using the actual PCM bits, assuming independent bit flips. A flip of the bit of weight $w = 2^k\Delta$ shifts the decoded level by $\pm w$, so with $e = \sum_k f_k\,\varepsilon_k w_k$ ($f_k \sim$ Bernoulli(p), $\varepsilon_k = \pm 1$ from the bit value), $q = x - x_q$ and $\mathbb{E}[e] = p\sum \varepsilon_k w_k$, $\mathbb{E}[e^2] = p\sum w_k^2 + p^2[(\sum \varepsilon_k w_k)^2 - \sum w_k^2]$:

$$\mathrm{MSE}(p) = \frac{1}{N}\sum_n \Big(q_n^2 - 2q_n\mathbb{E}[e_n] + \mathbb{E}[e_n^2]\Big) \;\approx\; \frac{\Delta^2}{12} + p\,\Delta^2\,\frac{4^n - 1}{3}. \tag{2}$$

The second term is dominated by the MSB: one MSB error moves a sample by $128\Delta = 1$, which, spread over 400 samples, raises the MSE by 2.5×10^{-3} — about 400 times the whole quantization MSE.

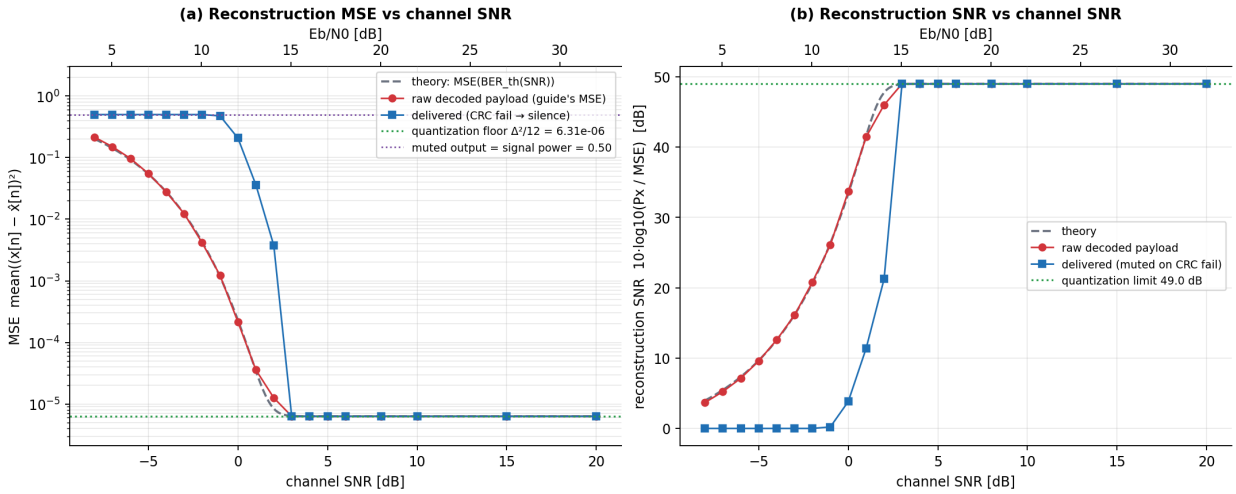


Figure 14: (a) Reconstruction MSE and (b) reconstruction SNR versus channel SNR. Markers: simulation; dashed: theory.

Table 3: Selected results of the SNR sweep (400 packets per row). PSR = packet success rate (sync word, CRC-16 and device ID all correct); $\mathrm{SNR}_{\mathrm{rec}}$ = reconstruction SNR of the raw payload.

SNR [dB]	E_b/N_0 [dB]	BER	PSR	MSE (raw)	$\mathrm{SNR}_{\mathrm{rec}}$ [dB]	MSE (delivered)
-6	6.0	6.71×10^{-2}	0.000	9.68×10^{-2}	7.1	5.00×10^{-1}
-4	8.0	2.06×10^{-2}	0.000	2.77×10^{-2}	12.6	5.00×10^{-1}
-2	10.0	3.23×10^{-3}	0.000	4.15×10^{-3}	20.8	5.00×10^{-1}
-1	11.0	8.92×10^{-4}	0.048	1.22×10^{-3}	26.1	4.76×10^{-1}
+0	12.0	1.73×10^{-4}	0.585	2.15×10^{-4}	33.7	2.08×10^{-1}
+1	13.0	2.33×10^{-5}	0.927	3.58×10^{-5}	41.5	3.63×10^{-2}
+2	14.0	1.51×10^{-6}	0.993	1.26×10^{-5}	46.0	3.76×10^{-3}
+3	15.0	0 (none seen)	1.000	6.31×10^{-6}	49.0	6.31×10^{-6}
+5	17.0	0 (none seen)	1.000	6.31×10^{-6}	49.0	6.31×10^{-6}

Reading the curves.

- **Above ≈ +3 dB (source-limited).** No bit errors; MSE = 6.31×10^{-6}, the quantization floor, and more channel SNR cannot improve it. Only more PCM bits would.
- **Around −1 to +2 dB (the cliff).** The BER passes from 10^{-6} to 10^{-3}; the channel-induced MSE equals the quantization MSE already at $p^* = 4.7 \times 10^{-6}$, i.e. at $E_b/N_0 = 13.6$ dB (SNR = +1.6 dB). The reconstruction SNR falls from 49 dB at +3 dB to 34 dB at 0 dB and 21 dB at −2 dB.
- **Below −2 dB (channel-limited).** BER above 3×10^{-3}, essentially every packet fails the CRC. Without the CRC the MSE keeps growing smoothly (the raw curve) toward ≈ 0.83, the value for random bits.

Packet success rate. The 90 % point is at SNR = +0.9 dB ($E_b/N_0 = 13.0$ dB), the 50 % point at -0.2 dB. In all 400 packets × 19 SNR values *no packet with undetected payload errors* passed the CRC. The delivered MSE (blue) is much worse than the raw MSE around the cliff: a CRC fail discards a packet with one bad MSB, which would have given a decent (but wrong) signal. The CRC trades a bounded, known loss (silence) for protection against rare catastrophic spikes — the right choice for a security-critical fob, the wrong one for an audio link, where one would rather conceal the error.

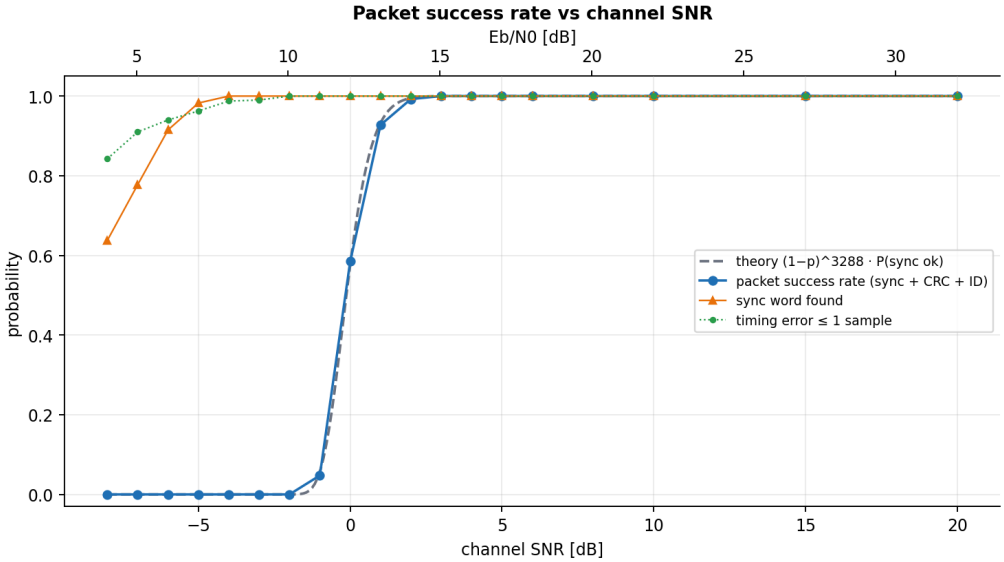


Figure 15: Packet success rate versus SNR; dashed: theory $(1-p)^{3288} \cdot P(\text{sync ok})$.

5 Link to Shannon's channel-capacity theorem

Shannon's theorem states that reliable communication at rate R over a channel of bandwidth B and signal-to-noise ratio S/N is possible *if and only if* $R \le C = B\log_2(1+S/N)$. We use the in-band SNR and $B = 4R_b = 38.4$ kHz. Equivalently, at spectral efficiency $\eta = R_b/B$ one needs $E_b/N_0 \ge (2^\eta - 1)/\eta$, which tends to -1.59 dB for $\eta \to 0$.

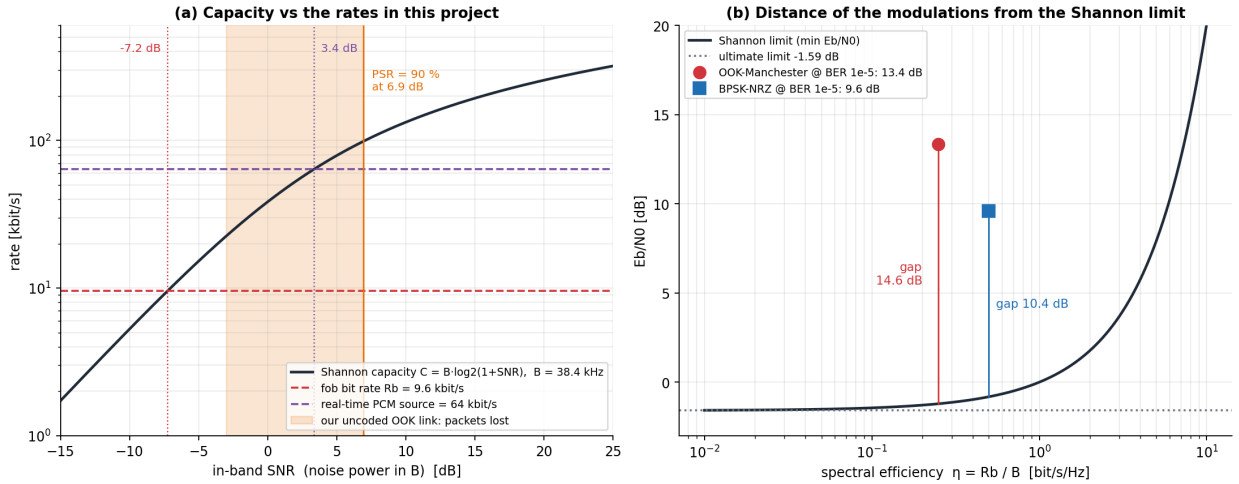


Figure 16: (a) Capacity of the $B = 38.4$ kHz channel against the fob bit rate and the real-time PCM rate. (b) E_b/N_0 needed for BER $= 10^{-5}$ by our two modulations compared with the Shannon limit.

Where is the channel the bottleneck? There are three different answers, all visible in the figures.

1. **Fundamental limit.** To carry $R_b = 9.6$ kbit/s in $B = 38.4$ kHz, any code and modulation needs an in-band SNR of at least -7.2 dB ($E_b/N_0 \ge -1.21$ dB). Below it the channel is the bottleneck for *every* system.
2. **Practical limit of our link.** Our uncoded OOK link reaches 90 % packet success only at an in-band SNR of 6.9 dB, and the MSE leaves the quantization floor below 7.6 dB. There the

Shannon capacity is 99 kbit/s, about ten times the bit rate used: the channel is *not* limiting for capacity reasons — our modulation and the absence of error-correcting code are. The gap to the Shannon limit at BER $= 10^{-5}$ is 14.6 dB for OOK-Manchester ($E_b/N_0 = 13.4$ dB needed against -1.21 dB) and 10.4 dB for BPSK-NRZ (9.6 dB against -0.82 dB). Forward error correction, soft detection and coding of the MSBs could win back most of it.

3. **Rate bottleneck of the source.** Streaming the PCM audio in real time needs $F_s \cdot n = 64$ kbit/s. Even an ideal coder needs an in-band SNR of 3.4 dB for this in 38.4 kHz, and our fob rate is 9.6 kbit/s: the 3200-bit clip needs 346 ms of airtime for 50 ms of audio, 6.7 times slower than real time. A fob therefore *buffers and bursts* the payload, which is why we model a packet and not a stream.

Conclusion of the discussion. At high SNR the *source* (quantization, 8 bits) is the bottleneck and the MSE is flat. In a window of only about 5 dB around an in-band SNR of 7 dB the *channel* takes over; the delivered MSE rises by almost five orders of magnitude (the raw MSE by nearly three), because one MSB error costs as much as 400 quantization noises. A designer should operate with a margin of several dB above that cliff (fading margin, Sec. 7) and, to approach the Shannon limit, spend bits on error protection of the MSBs instead of on more quantization levels.

6 Security layer: rolling code

The hop code is a 32-bit block encrypted with a 528-round non-linear-feedback shift-register cipher modelled on KeeLoq (64-bit key shared by fob and car). The plaintext is [4-bit command | 12-bit discrimination (low bits of the serial number) | 16-bit counter]. Every button press increments the counter. The car decrypts the hop code, checks the discrimination and command bits (a random hop code passes with probability 2^{-16}) and compares the counter with the last accepted one (Fig. 17). This is a teaching simulation, not a secure implementation.

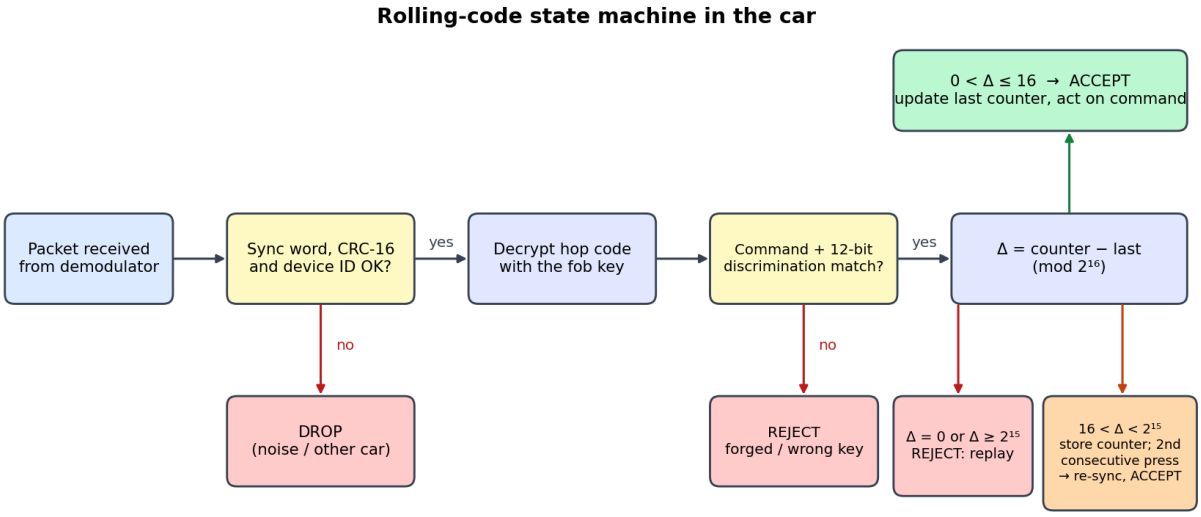


Figure 17: Receiver decision logic. Δ = counter − last counter (mod 2^{16}), look-ahead window 16.

We ran nine events through the *full waveform chain* (modulation, channel, blind demodulation, frame parser, rolling-code check); Table 4 and Fig. 18 show the results. In event 2 an attacker re-transmits a recording of event 1: the packet arrives perfectly (sync, CRC and ID all OK) and is nevertheless rejected because its counter was already used — the security is in the counter, not in the radio link.

Table 4: Security demonstration (SNR 8 dB; event 6 at −6 dB).

#	who	event	pkt ctr	car ctr	decision	reason
1	owner	UNLOCK pressed next to the car	1	1	**ACCEPT**	valid rolling code
2	attacker	recorded UNLOCK signal re-transmitted (replay attack)	1	1	REJECT	replay: counter already used
3	owner	3 presses out of range, then LOCK in range	5	5	**ACCEPT**	valid rolling code
4	stranger	unpaired fob of another car	1	5	REJECT	ignored: device ID not paired with this car
5	attacker	forged packet: sniffed ID, valid CRC, guessed hop code	–	5	REJECT	hop code failed authentication (forged / wrong key)
6	owner	UNLOCK pressed at SNR = -6 dB (too noisy)	6	5	REJECT	dropped at link layer: sync word + CRC check failed
7	owner	user presses again, link is good	7	7	**ACCEPT**	valid rolling code
8	owner	20 blind presses, then 1st press in range	28	7	REJECT	counter outside look-ahead window - press again to re-sync
9	owner	2nd consecutive press → re-synchronisation	29	29	**ACCEPT**	re-synchronised (two consecutive counters)

#	who	what happens	counter	car last	link	decision	reason
1	owner	UNLOCK pressed next to the car	1	1	OK	ACCEPT	valid rolling code
2	attacker	recorded UNLOCK signal re-transmitted (replay attack)	1	1	OK	REJECT	replay: counter already used
3	owner	3 presses out of range, then LOCK in range	5	5	OK	ACCEPT	valid rolling code
4	stranger	unpaired fob of another car	1	5	FAIL	REJECT	ignored: device ID not paired with this car
5	attacker	forged packet: sniffed ID, valid CRC, guessed hop code	-	5	OK	REJECT	hop code failed authentication (forged / wrong key)
6	owner	UNLOCK pressed at SNR = -6 dB (too noisy)	6	5	FAIL	REJECT	dropped at link layer: sync word + CRC check failed
7	owner	user presses again, link is good	7	7	OK	ACCEPT	valid rolling code
8	owner	20 blind presses, then 1st press in range	28	7	OK	REJECT	counter outside look-ahead window - press again to re-sync
9	owner	2nd consecutive press -> re-synchronisation	29	29	OK	ACCEPT	re-synchronised (two consecutive counters)

Figure 18: Security demonstration timeline.

7 Range: from SNR to distance

A log-distance link budget turns the SNR axis into metres: free-space loss at 1 m for 433.92 MHz (25.2 dB), path-loss exponent $n = 3.5$ (obstructed), 15 dB extra body/car shadowing, fob EIRP −15 dBm, receiver noise floor -118.2 dBm (kTB over 38.4 kHz plus 10 dB noise figure). Fading is modelled as flat and constant during one 346 ms packet (block fading): Rayleigh ($|h|^2$ exponential, no line of sight, like a parking garage) and Rician with $K = 6$ dB (line of sight). These link-budget numbers are *illustrative assumptions*, not measurements.

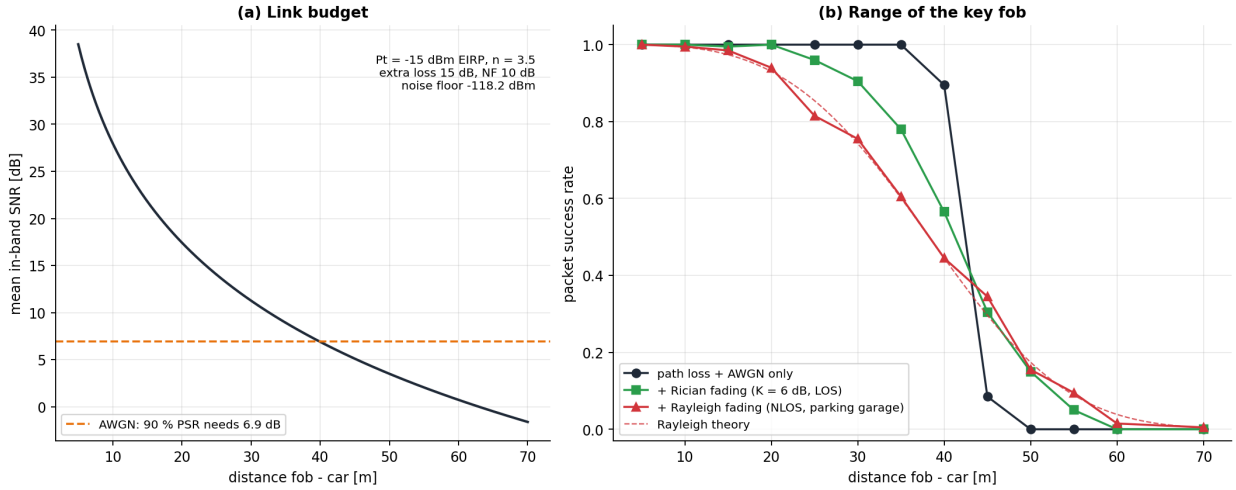


Figure 19: (a) Mean in-band SNR versus distance. (b) Packet success rate versus distance for three channels; the dashed line is the analytic Rayleigh average of the AWGN packet-success curve.

Table 5: Packet success rate versus distance (200 packets per point).

Distance [m]	$\mathrm{SNR}_{\text{in-band}}$ [dB]	AWGN only	Rician K=6 dB	Rayleigh
10	28.0	1.00	1.00	0.99
20	17.4	1.00	1.00	0.94
30	11.3	1.00	0.91	0.76
40	6.9	0.90	0.56	0.45
50	3.5	0.00	0.15	0.15
60	0.7	0.00	0.00	0.01

The 90 % range is about 40 m without fading, 30 m with a line-of-sight component and 22 m with Rayleigh fading. Fading therefore costs almost half of the range: a deep fade of a few dB is enough to lose a 3320-bit packet whose CRC allows no error at all. This is why real fobs repeat the frame several times, and why the user simply presses the button again.

8 Verification, assumptions and limitations

Verification. (i) CRC-16 reproduces the standard check value `0x29B1`; (ii) KeeLoq-style encrypt/decrypt, PCM, quantizer and Manchester round trips, the clean-channel loop-back, rejection under heavy noise, replay/window/forgery rules are unit-tested (`tests/`, 12 tests, `python -m unittest discover -s tests`); (iii) the simulated BER matches theory for both OOK and BPSK (Fig. 13) and the simulated MSE matches the analytic MSE(p) (Fig. 14); (iv) no undetected-error packet was observed. All results are reproducible with the seed in `Config`.

A pitfall we fixed. Our first channel rotated the carrier phase with a Hilbert transform. With the scaled carrier ($f_c/R_b = 4$) the OOK spectrum is not narrow compared with f_c, the transform leaked energy into the "off" chips and the BER was about 1.4–1.6 times too high. The final channel uses the exact quadrature pair $\mathrm{Re}\{h\,(x_I + jx_Q)\}$, after which the simulation matches theory.

Assumptions and limitations. Scaled carrier (valid for error performance, see Sec. 2). Perfect sample-rate synchronisation between transmitter and receiver (no oscillator drift or frequency offset). No AGC, interference or multipath delay spread; flat block fading. KeeLoq-style cipher written from the published algorithm structure; it is not validated against official test vectors and

is not meant to protect anything real. Monte-Carlo confidence: a point with k errors has a relative standard error of about $1/\sqrt{k}$; points with fewer than ≈30 errors (BER below $\approx 2 \times 10^{-5}$) are indicative only. The link-budget parameters are assumptions.

9 Conclusion

We built and validated a complete, reproducible simulation of a key-fob link in which every block of the guideline chain is present and the physical-layer results agree with theory. The reconstruction quality is flat at the 8-bit quantization limit of 48.99 dB until the channel SNR approaches about 7.6 dB in-band, then collapses within a few dB; at the 90 % packet-success point the link is 14.2 dB from the Shannon limit, so coding rather than capacity limits it. The rolling code makes recorded signals useless, and the link budget shows that fading, not noise alone, sets the useful range.

How to run and repository layout

```
pip install -r requirements.txt
python main.py            # full run (about 2-3 minutes) -> figures/, results/
python main.py --quick    # fast preview
python -m unittest discover -s tests -v
python report/build_report.py   # rebuilds report.pdf (needs LaTeX)
```

```
main.py             entry point: runs the complete chain
functions/          config, source, quantizer, pcm, framing, rolling_code,
    line_coding,
                    modulation, channel, demodulation, receiver, reconstruction,
                    metrics, analysis, plots_*.py
figures/            all saved plots (01 ... 19)
results/            metrics.json and CSV tables of the sweeps
report.ipynb        the report as an executed notebook      report.pdf   this
    report
tests/              unit tests                              README.md
```

References

[1] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, 27, 1948.

[2] J. G. Proakis and M. Salehi, *Digital Communications*, 5th ed., McGraw-Hill, 2008 (error probability of OOK/non-coherent and BPSK detection, Shannon limit).

[3] Microchip Technology, KeeLoq code-hopping encoder data sheets (HCS series) — structure of the hop code and counter-window logic.